

Verso gli array.

- Passo 1: un insieme di variabili

```
int alt0=1, alt1=3, alt2=5, alt3=7;
media= (alt0 +alt1 +alt2 +alt3)/4;
```

- Passo 2: introduco gli array

```
int alt[]={1,3,5,7}; //inizializzazione array
media= (alt[0]+alt[1]+alt[2]+alt[3])/4;
```

- Passo 3: introduco un indice

```
ind=0;
media = alt[ind];
ind++;
media = media + alt[ind];
ind++;
media = media + alt[ind];
ind++;
media = media + alt[ind];
media= media/ind;
...
- Passo 4: loop sull'array
```C
```

```
int ind, media=0;
for(ind=0; ind<4; ind++)
 media += alt[ind];
media/=ind;
```

---

## Media

Scrivere una funzione che riceve in ingresso un array di interi e restituisce la media dei valori

```
float avgVet(int par[], int len) {
 int media, ind;
 for (ind = 0, media = 0; ind < len; ind++)
 media = media + par[ind];
 return ((float)media) / ind;
}
```

---

## Minimo di un array

Scrivere una funzione che riceve un vettore di interi e ne restituisce il minimo

```
int minimo(int vet[], int len) {
 int i;
 int min = vet[0];
 for (i = 1; i < len; i++) {
 if (min > vet[i])
 min = vet[i];
 }
 return min;
}
```

---

## checkOrder

Scrivere una funzione che riceve un vettore di numeri interi e verifica se gli elementi sono in ordine strettamente crescente restituendo un valore booleano

```
#include <stdbool.h>
bool checkOrder(int vet[], int len){
 for (int i = 0; i < len-1; i++) {
 if(vet[i]>vet[i+1])
 return false;
 }
 return true;
}
```

---

## Shift & Rotate

```
//Left Shift
void lsh(int v[], int len){
 for(int i=1;i<len;i++){
 v[i-1]=v[i];
 }
 v[4]=0;
}

//Right shift
void lsr(int v[], int len){
 int pippo;
 pippo=v[0];
 for(int i=1;i<len;i++){
 v[i-1]=v[i];
 }
 v[len-1]=pippo;
}
```

```

 }
 v[4]=pippo;
}

//Left Shift & Rotate
void lsr(int v[], int len){
 int pippo;
 pippo=v[0];
 for(int i=1;i<len;i++){
 v[i-1]=v[i];
 }
 v[4]=pippo;
}

```

## Riempi di casuali

```

//Riempire vettore di numeri casuali
void riempiVet(int vet[], int len, int max, int min) {
 int diff = max - min + 1;
 srand(time(NULL));
 for (int i = 0; i < len; i++) {
 vet[i] = rand() % diff + min;
 }
}

```

## Occorrenze in un array

Scrivere una funzione che riceve un array di dimensione DIM e un numero N;  
restituire il numero di occorrenze(quante volte è presente) del valore N nell'array

```

int occorrenze(int vet[], int len, int n) {
 int occ = 0;
 int i;
 for (i = 0; i < len; i++) {
 if (vet[i] == n)
 occ++;
 }
 return occ;
}

```

## Riempire un array con multipli di N non divisibili per D

Dato un array vuoto, due numeri N e D, riempire l'array con i multipli di N non divisibili per D

```

#define M 20
void multipli(int array[M], int N, int D) {
 int numero = 1;
 for (int i=0; i<M; numero++) {
 if (numero * N % D != 0) {
 array[i] = numero * N;
 i++;
 }
 }
}

```

---

## Palindromo

```

// palindromicità
bool IsPal(int v[], int len){
 int iMin=0, iMax=len-1;
 for(;iMin<iMax;iMin++,iMax--){
 if(v[iMin]!=v[iMax])
 return false;
 }
 return true;
}

```

## Reverse

```

// Reverse un array
void reverse(int v[], int len){
 int iMin=0, iMax=len-1, temp;
 for(;iMin<=iMax;iMin++,iMax--){
 temp=v[iMin];
 v[iMin]=v[iMax];
 v[iMax]=temp;
 }
}

```

## Eliminazione con compattamento

Eliminare un valore e compattare con SHL

```

/*
 * Dato un array eliminare un valore, compattandolo e inserendo un valore in fondo(0)
 */
void f(int arr[], int len, int val) {
 int i;
 for (i = 0; i < len; i++) {
 if (arr[i] == val) {
 break;
 }
 }
}

```

```

 }
}
for (int j = i; j < len - 1; j++) {
 arr[j] = arr[j + 1];
}
arr[len-1]=0;
}

int main() {
 int arr[] = {1, 2, 3, 4, 5};
 int len = sizeof(arr) / sizeof(arr[0]);
 f(arr,len,3);
 return 0;
}

```

---

## Compattatore di zeri

Dato un vettore di numeri interi positivi, sposta gli elementi pari a zero in fondo al vettore. Restituisce il numero di elementi diversi da zero.

```

void pushZerosToEnd(int arr[], int n) {
 int count = 0; // Contatore elementi non-zero
 int ret;

 // Se l'elemento è non-zero, si copia l'elemento corrente nell'indice 'count'
 // Con gli zeri si avanza solo con l'indice corrente creando uno sfalsamento
 for (int i = 0; i < n; i++) {
 if (arr[i] != 0)
 arr[count++] = arr[i];
 }
 // Alla fine del ciclo tutti gli elementi diversi da zeri risultano
 // spostati nella prima parte dell'array.
 // Rimane da riempire con zeri la rimanente parte
 ret = count;
 while (count < n)
 arr[count++] = 0;

 return ret;
}

```